

Presentation–Abstraction–Control (PAC)

Hospital Monitoring System Application

Mohamed Khider University — Biskra
Computer Science Department
3rd Year Software Engineering | 2025/2026
Module: Software Design | Teacher: Ouaar Hanane
student : derbali oussama

Practical Work N°3-2

Contents

1	Introduction and Context	3
1.1	Project Overview	3
1.2	Objectives	3
2	The PAC Pattern	3
2.1	What is PAC?	3
2.2	PAC Components	3
2.2.1	PACAbstraction	3
2.2.2	PACControl	4
2.2.3	PACPresentation	4
2.3	Control-to-Control Communication	4
3	System Architecture	4
3.1	System Overview	4
3.2	Agent Hierarchy	5
4	Agent Descriptions	5
4.1	PatientAgent	5
4.1.1	Purpose	5
4.1.2	Components	5
4.1.3	Key Interactions	6
4.2	DeviceAgent	6
4.2.1	Purpose	6
4.2.2	Components	6
4.2.3	Key Interactions	7
4.3	NurseAgent	7
4.3.1	Purpose	7
4.3.2	Components	7
4.3.3	Key Interactions	8

4.4	DoctorAgent	8
4.4.1	Purpose	8
4.4.2	Components	8
4.4.3	Key Interactions	9
5	UML Diagrams	9
5.1	Class Diagram	9
5.2	Sequence Diagram	12
5.2.1	Device Recording Vital Signs	13
5.2.2	Patient Calling Nurse	13
5.2.3	Patient Calling Doctor	13
5.2.4	Doctor Modifying Treatment	13
5.2.5	Nurse Completing Tasks	14
5.2.6	Device Control Operations	14
6	Control-to-Control Communication	14
6.1	Communication Mechanism	14
6.2	Communication Matrix	14
6.3	Message Types	14
6.3.1	PatientControl Messages	14
6.3.2	DeviceControl Messages	15
6.3.3	NurseControl Messages	15
6.3.4	DoctorControl Messages	15
7	Implementation Details	15
7.1	Technology Stack	15
7.2	Project Structure	15
7.3	Base PAC Framework	16
7.4	State Management	16
8	Key Features	16
8.1	Real-time Monitoring	16
8.2	Alert Propagation	16
8.3	Patient-Initiated Requests	17
8.4	Treatment Modification	17
8.5	Simulating Alerts	17
8.6	Device Operations	17
9	Benefits of PAC Pattern	18
9.1	Separation of Concerns	18
9.2	Modularity	18
9.3	Scalability	18
9.4	Maintainability	18
10	Conclusion	19
A	Implementation Example: PACControl Base Class	19
B	References	20

1 Introduction and Context

1.1 Project Overview

The PAC Hospital Monitoring System is a smart hospital monitoring application built using the Presentation-Abstraction-Control (PAC) architectural pattern. The system demonstrates how the PAC pattern can be used to create a distributed, hierarchical control system for monitoring patients, medical devices, nurses, and doctors in a hospital environment.

1.2 Objectives

- Implement the PAC architectural pattern in a Flutter application
- Create four distinct PAC agents: Patient, Device, Nurse, and Doctor
- Demonstrate control-to-control communication between agents
- Show how the PAC pattern enables separation of concerns and modular design
- Provide a working demonstration of agent interactions

2 The PAC Pattern

2.1 What is PAC?

The Presentation-Abstraction-Control (PAC) pattern is an architectural pattern used for designing interactive software systems. It organizes the system into a hierarchy of cooperating agents, each consisting of three components:

- **Presentation:** Handles the user interface and user interactions
- **Abstraction:** Manages the data and domain logic
- **Control:** Mediates between Presentation and Abstraction, handles communication with other agents

2.2 PAC Components

2.2.1 PACAbstraction

The abstraction component is responsible for:

- Maintaining the agent's internal state
- Providing access to domain data
- Implementing business logic
- No knowledge of presentation or other agents

2.2.2 PACControl

The control component is responsible for:

- Mediating between presentation and abstraction
- Handling control-to-control communication
- Processing user actions from presentation
- Notifying presentation of state changes

2.2.3 PACPresentation

The presentation component is responsible for:

- Displaying the agent's UI
- Handling user interactions
- Forwarding user actions to control
- Updating UI when notified by control

2.3 Control-to-Control Communication

A key feature of PAC is that controls can communicate directly with each other. This enables:

- Distributed decision-making
- Hierarchical coordination
- Event-driven interactions between agents
- Loose coupling between agents

3 System Architecture

3.1 System Overview

The PAC Hospital Monitoring System consists of four main agents:

1. **PatientAgent**: Monitors patient vitals, treatment plans, and patient requests
2. **DeviceAgent**: Manages medical devices, records readings, and generates alerts
3. **NurseAgent**: Handles nurse tasks, patient assignments, and duty status
4. **DoctorAgent**: Manages doctor orders, treatment modifications, and consultations

3.2 Agent Hierarchy

The agents work together in a coordinated hierarchy where:

- **PatientAgent** receives vital sign updates from DeviceAgent
- **PatientAgent** communicates patient requests to NurseAgent and DoctorAgent
- **DeviceAgent** sends alerts to both PatientAgent and NurseAgent
- **DoctorAgent** sends treatment updates back to PatientAgent
- **NurseAgent** manages tasks and patient assignments

4 Agent Descriptions

4.1 PatientAgent

4.1.1 Purpose

The PatientAgent manages patient information including vital signs, treatment plans, and patient-initiated requests.

4.1.2 Components

PatientAbstraction:

- Stores Patient domain model
- Manages vital signs history
- Tracks treatment plans
- Detects critical vital signs

PatientControl:

- Receives vital sign updates from DeviceControl
- Forwards nurse requests to NurseControl
- Forwards doctor requests to DoctorControl
- Notifies presentation of state changes

PatientPresentation:

- Displays patient name, age, condition
- Shows current vital signs
- Displays treatment plan
- Provides "Call Nurse" and "Call Doctor" buttons
- Shows CRITICAL badge when vitals are abnormal

4.1.3 Key Interactions

- Receives vital sign updates from DeviceControl
- Sends nurse requests to NurseControl
- Sends doctor requests to DoctorControl
- Receives treatment updates from DoctorControl

4.2 DeviceAgent

4.2.1 Purpose

The DeviceAgent manages medical devices, records patient readings, and generates alerts when readings exceed thresholds.

4.2.2 Components

DeviceAbstraction:

- Stores MedicalDevice domain model
- Manages device readings history
- Tracks device status (idle, active, maintenance)
- Detects alert conditions
- Manages device settings

DeviceControl:

- Records new device readings
- Checks for alert conditions
- Forwards alerts to PatientControl and NurseControl
- Handles device operations (start, stop, calibrate)

DevicePresentation:

- Displays device name, type, status
- Shows latest reading
- Shows assignment status
- Shows ALERT badge when readings are critical
- Provides Start, Stop, Calibrate buttons

4.2.3 Key Interactions

- Sends vital sign updates to PatientControl
- Sends device alerts to NurseControl
- Receives patient assignments from external system

4.3 NurseAgent

4.3.1 Purpose

The NurseAgent manages nurse tasks, patient assignments, and duty status. It receives alerts and requests from other agents and creates tasks accordingly.

4.3.2 Components

NurseAbstraction:

- Stores Nurse domain model
- Manages task list
- Tracks patient assignments
- Manages duty status
- Identifies urgent tasks

NurseControl:

- Receives patient alerts from PatientControl
- Receives device alerts from DeviceControl
- Creates tasks based on alerts
- Handles task completion
- Manages patient assignments

NursePresentation:

- Displays nurse name, department, duty status
- Shows assigned patients
- Shows pending tasks
- Highlights urgent tasks
- Provides duty toggle switch
- Provides task completion controls

4.3.3 Key Interactions

- Receives nurse requests from PatientControl
- Receives device alerts from DeviceControl
- Communicates with PatientControl and DoctorControl

4.4 DoctorAgent

4.4.1 Purpose

The DoctorAgent manages doctor orders, treatment modifications, and consultations. It can modify patient treatment plans and review patient alerts.

4.4.2 Components

DoctorAbstraction:

- Stores Doctor domain model
- Manages order list
- Tracks patient assignments
- Manages availability status
- Tracks pending orders

DoctorControl:

- Receives doctor requests from PatientControl
- Receives patient alerts from PatientControl
- Creates consultation orders
- Modifies patient treatment plans
- Handles order approval/rejection

DoctorPresentation:

- Displays doctor name, specialization, availability
- Shows assigned patients
- Shows pending orders
- Provides order approval/rejection controls
- Provides availability toggle switch

4.4.3 Key Interactions

- Receives doctor requests from PatientControl
- Receives patient alerts from PatientControl
- Sends treatment updates to PatientControl
- Communicates with NurseControl

5 UML Diagrams

5.1 Class Diagram

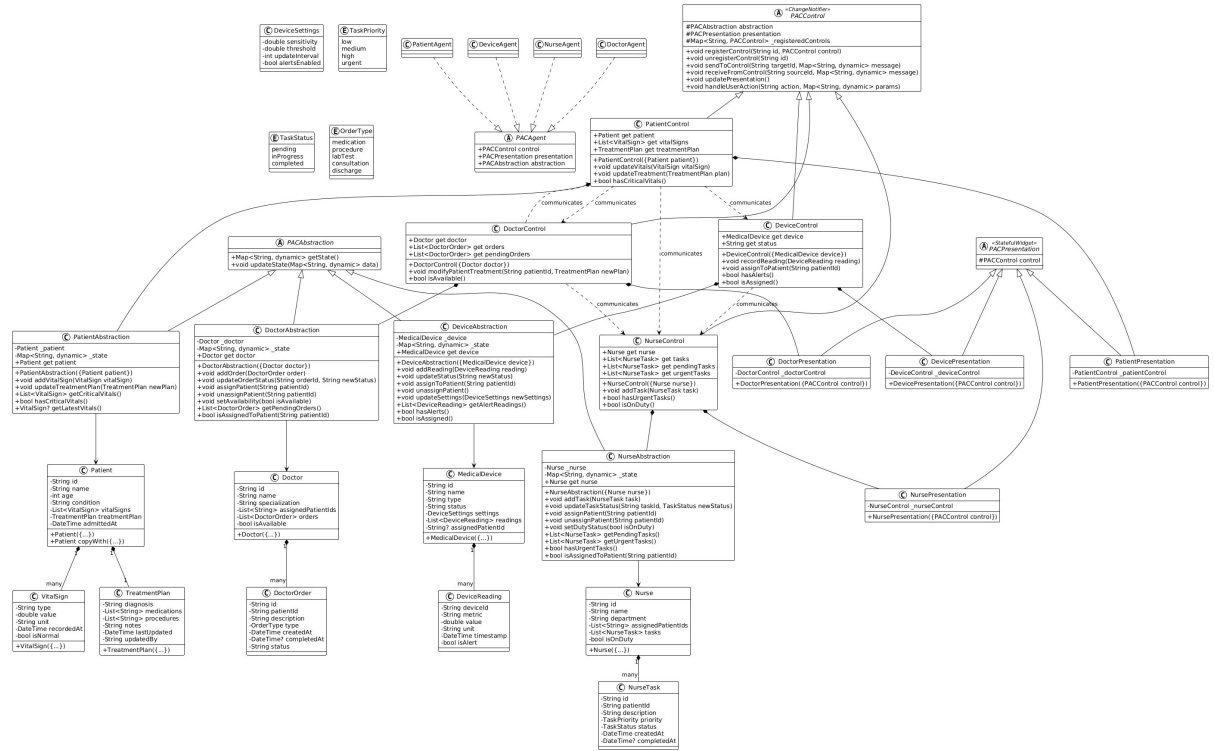


Figure 1: PAC Class Diagram for Hospital Monitoring System. Shows complete class structure with domain models (Patient, Doctor, Device, Nurse, MedicalDevice, VitalSign, TreatmentPlan, DoctorOrder, DeviceReading, NurseTask), PAC base classes (PACAgent, PACControl, PACAbstraction, PACPresentation), and all four agents (PatientAgent, DoctorAgent, DeviceAgent, NurseAgent) with their three components each. Relationships indicate composition, aggregation, and dependency connections between classes.

The Class Diagram illustrates:

- **Domain Models:** Patient, Doctor, Device, Nurse, MedicalDevice, VitalSign, TreatmentPlan, DoctorOrder, DeviceReading, NurseTask
- **PAC Base Classes:** PACAgent, PACControl (extends ChangeNotifier), PACAbstraction, PACPresentation

- **Agent Components:** Each of the four agents (Patient, Device, Nurse, Doctor) has three components following the PAC pattern
- **Relationships:** Composition, aggregation, and dependency relationships showing how components interact

5.2 Sequence Diagram

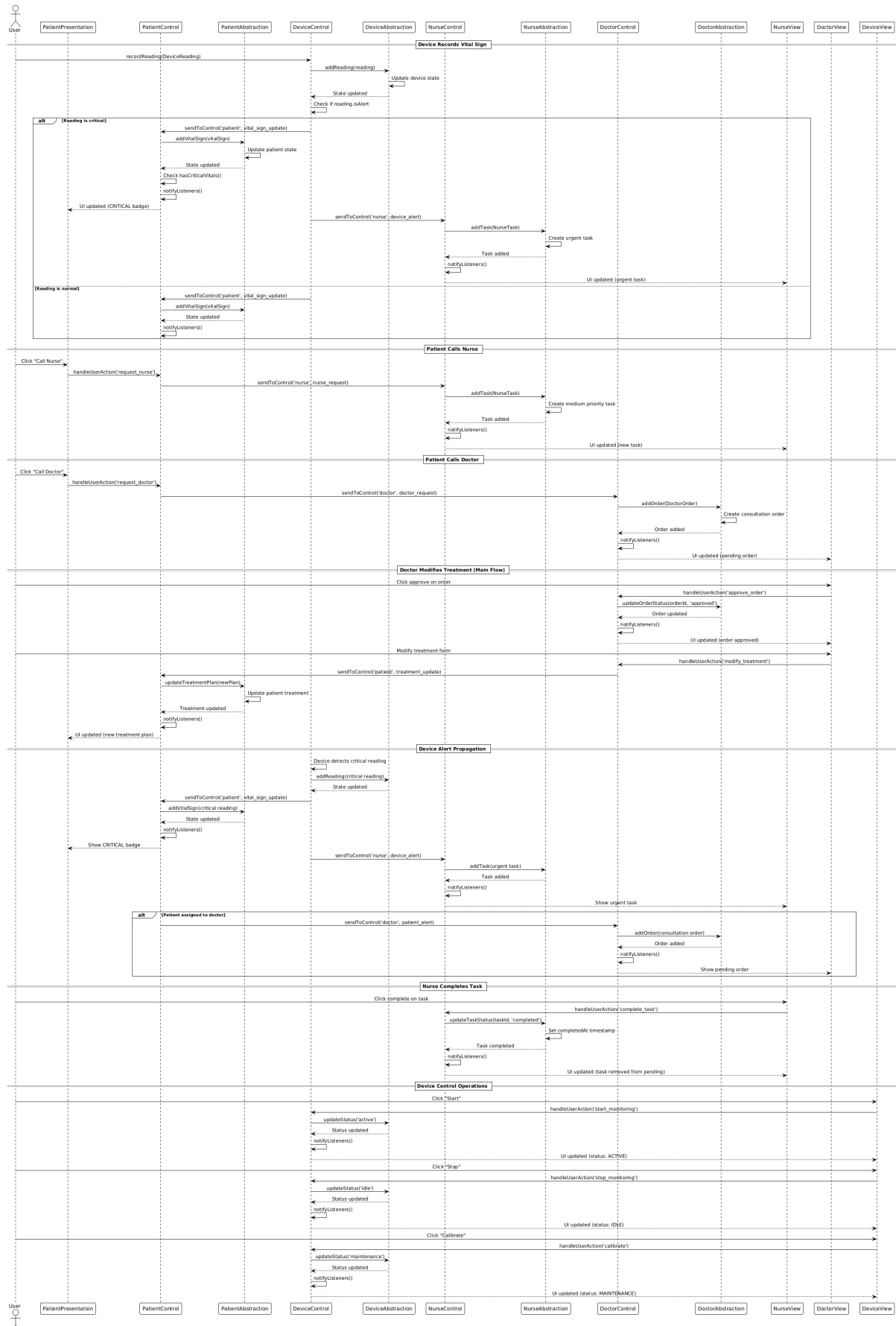


Figure 2: PAC Sequence Diagram showing multiple interaction flows in the Hospital Monitoring System. Demonstrates: (1) Device Recording Vital Signs and updating Patient state, (2) Patient calling Nurse/Doctor, (3) Nurse completing tasks, (4) Doctor modifying treatment, (5) Device control operations, and (6) Alert propagation across agents. Shows

The Sequence Diagram demonstrates the following key flows:

5.2.1 Device Recording Vital Signs

1. Device records a vital sign reading
2. DeviceControl detects if reading is critical
3. DeviceControl sends vital sign update to PatientControl
4. PatientControl updates PatientAbstraction with new reading
5. PatientPresentation rebuilds showing updated vital signs
6. If critical, CRITICAL badge is displayed

5.2.2 Patient Calling Nurse

1. Patient clicks "Call Nurse" button in PatientPresentation
2. PatientControl receives the request
3. PatientControl sends nurse request message to NurseControl
4. NurseControl creates new task in NurseAbstraction
5. NursePresentation updates to show new pending task

5.2.3 Patient Calling Doctor

1. Patient clicks "Call Doctor" button
2. PatientControl receives request
3. PatientControl sends doctor request to DoctorControl
4. DoctorControl creates consultation order
5. DoctorPresentation displays new pending order

5.2.4 Doctor Modifying Treatment

1. Doctor approves consultation order in DoctorPresentation
2. DoctorControl modifies treatment plan in DoctorAbstraction
3. DoctorControl sends treatment update to PatientControl
4. PatientControl updates PatientAbstraction with new plan
5. PatientPresentation shows updated treatment plan

5.2.5 Nurse Completing Tasks

1. Nurse marks task as complete
2. NurseControl updates NurseAbstraction
3. NursePresentation removes completed task from list
4. Notification sent back to requesting agent

5.2.6 Device Control Operations

1. User clicks Start/Stop/Calibrate button
2. DevicePresentation forwards action to DeviceControl
3. DeviceControl updates device status
4. DevicePresentation displays new status

6 Control-to-Control Communication

6.1 Communication Mechanism

The PAC pattern enables direct communication between control components through a registration system:

- Each control maintains a registry of other controls
- Controls register with each other during initialization
- Communication happens via message passing
- Messages contain a type and payload data

6.2 Communication Matrix

Table 1: Control-to-Control Communication Matrix

From / To	Patient	Device	Nurse	Doctor
Patient	—	—	nurse_request	doctor_request
Device	vital_sign_update	—	device_alert	—
Nurse	—	—	—	—
Doctor	treatment_update	—	—	—

6.3 Message Types

6.3.1 PatientControl Messages

- **vital_sign_update**: Sent from DeviceControl when new reading recorded
- **treatment_update**: Sent from DoctorControl when treatment modified

6.3.2 DeviceControl Messages

- `vital_sign_update`: Sent to PatientControl with new device reading
- `device_alert`: Sent to NurseControl when reading exceeds threshold

6.3.3 NurseControl Messages

- `nurse_request`: Received from PatientControl when patient calls nurse
- `device_alert`: Received from DeviceControl when alert generated

6.3.4 DoctorControl Messages

- `doctor_request`: Received from PatientControl when patient calls doctor
- `patient_alert`: Received from PatientControl when condition critical

7 Implementation Details

7.1 Technology Stack

- **Language**: Dart
- **Framework**: Flutter
- **State Management**: ChangeNotifier (built into Flutter)
- **Platform**: Cross-platform (Web, iOS, Android)

7.2 Project Structure

Listing 1: Project directory structure

```
lib/  
  main.dart                # Entry point  
  models/                  # Domain models  
    patient.dart  
    device.dart  
    nurse.dart  
    doctor.dart  
  pac/                      # PAC framework  
    core/  
      pac_agent.dart        # Base PAC classes  
    agents/  
      patient/  
        patient_abstraction.dart  
        patient_control.dart  
        patient_presentation.dart  
      device/  
      nurse/  
      doctor/  
  pubspec.yaml
```

7.3 Base PAC Framework

The base PAC framework is implemented in `pac/core/pac_agent.dart`:

- **PACAgent**: Base class for all agents
- **PACAbstraction**: Abstract class for abstraction components
- **PACControl**: Abstract class extending `ChangeNotifier` for control components
- **PACPresentation**: Abstract class for presentation components

7.4 State Management

The system uses Flutter's `ChangeNotifier` pattern:

- All control components extend `ChangeNotifier`
- Presentation components listen to their control
- When state changes, control calls `notifyListeners()`
- Presentation rebuilds when notified

8 Key Features

8.1 Real-time Monitoring

The system demonstrates real-time monitoring through:

- Device readings triggering immediate updates
- Critical alerts propagating through the system
- UI updates reflecting state changes instantly

8.2 Alert Propagation

When a device records a critical reading:

1. `DeviceControl` detects the alert
2. `DeviceControl` sends vital sign update to `PatientControl`
3. `PatientControl` updates patient state and shows **CRITICAL** badge
4. `DeviceControl` sends device alert to `NurseControl`
5. `NurseControl` creates urgent task
6. If patient assigned to doctor, `DoctorControl` receives alert

8.3 Patient-Initiated Requests

Patients can initiate requests:

- **Call Nurse:** Creates a medium-priority task for the nurse
- **Call Doctor:** Creates a consultation order for the doctor

8.4 Treatment Modification

Doctors can modify patient treatment plans:

1. Doctor approves consultation order
2. Doctor modifies treatment plan
3. DoctorControl sends treatment update to PatientControl
4. PatientControl updates patient state
5. Patient presentation shows new treatment plan

8.5 Simulating Alerts

The application includes a "Simulate Alert" button in the app bar. Clicking this:

- Records a critical heart rate reading (130 bpm)
- Triggers alert propagation
- Creates urgent task for nurse
- Shows CRITICAL badge on patient
- Shows ALERT badge on device

8.6 Device Operations

The device presentation provides three operations:

- **Start:** Changes device status to 'active'
- **Stop:** Changes device status to 'idle'
- **Calibrate:** Changes device status to 'maintenance'

After calibration, the device can be started again.

9 Benefits of PAC Pattern

9.1 Separation of Concerns

Each agent's components have clear responsibilities:

- Presentation handles only UI
- Abstraction handles only data and business logic
- Control handles only coordination and communication

9.2 Modularity

Agents are independent and can be:

- Developed separately
- Tested independently
- Reused in different contexts
- Modified without affecting other agents

9.3 Scalability

The hierarchical nature of PAC supports:

- Adding new agents easily
- Creating sub-agents for complex functionality
- Distributed deployment
- Horizontal scaling

9.4 Maintainability

The pattern promotes:

- Clear code organization
- Easy to understand structure
- Isolated changes
- Testable components

10 Conclusion

The PAC Hospital Monitoring System successfully demonstrates the Presentation-Abstraction-Control pattern in a practical application. The system shows how:

- The PAC pattern enables clean separation of concerns
- Control-to-control communication enables distributed decision-making
- The pattern supports modular, maintainable architecture
- The system can be extended with additional agents easily

The implementation provides a working example of how PAC can be used to build complex, interactive systems with hierarchical control structures. The four agents (Patient, Device, Nurse, Doctor) work together through control-to-control communication to provide a comprehensive hospital monitoring solution.

A Implementation Example: PACControl Base Class

Listing 2: Base PAC Control class structure

```
abstract class PACControl extends ChangeNotifier {
    final PACAbstraction abstraction;
    final Map<String, PACControl> registeredControls = {};

    PACControl(this.abstraction);

    // Register another control for communication
    void registerControl(String name, PACControl control) {
        registeredControls[name] = control;
    }

    // Send message to another control
    void sendMessage(String targetControl,
                     String messageType,
                     dynamic data) {
        final target = registeredControls[targetControl];
        if (target != null) {
            target.receiveMessage(messageType, data);
        }
    }

    // Receive message from another control
    void receiveMessage(String messageType, dynamic data);
}
```

B References

- Buschmann, F., et al. (1996). *Pattern-Oriented Software Architecture, Volume 1*. Wiley.
- Flutter Documentation: <https://flutter.dev/docs>
- Dart Language: <https://dart.dev/guides>